Module IV: Graph Neural Network

What Are Graph Neural Networks?

Graph neural networks apply the predictive power of deep learning to rich data structures that depict objects and their relationships as points connected by lines in a graph.

In GNNs, data points are called nodes, which are linked by lines — called edges — with elements expressed mathematically so machine learning algorithms can make useful predictions at the level of nodes, edges or entire graphs.

GNNs typically adopt a <graph-in, graph-out= architecture. This means that these model types accept a graph as input, with information loaded into its nodes, edges and global context. The models progressively transform these embeddings without changing the connectivity of the input graph. Embeddings represent the nodes as node embeddings, and the vertices as vertex embeddings. These embeddings allow the model to learn what types of nodes occur and where in the graph as well as the types and locations of edges.

Key Aspects of GNNs:

1 **Structure:**

GNNs operate on data comprising nodes (entities) and edges (relationships).

2 **Message Passing**

The core process where nodes communicate with neighbors to gather structural information.

3 **Graph Embedding:**

GNNs create a vector representation of nodes (node embedding) that includes information about their position and neighbors.

4 **Capabilities:**

They excel in tasks like **node classification** (predicting a node type), **link prediction** (predicting relationships), and **graph classification** (labeling an entire graph).

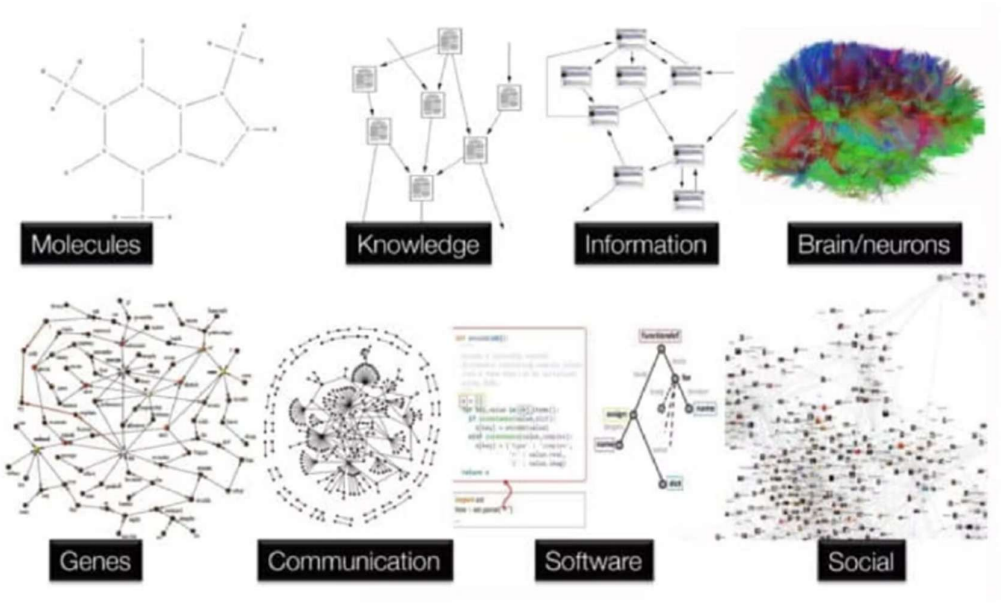


Fig :How graph data looks like

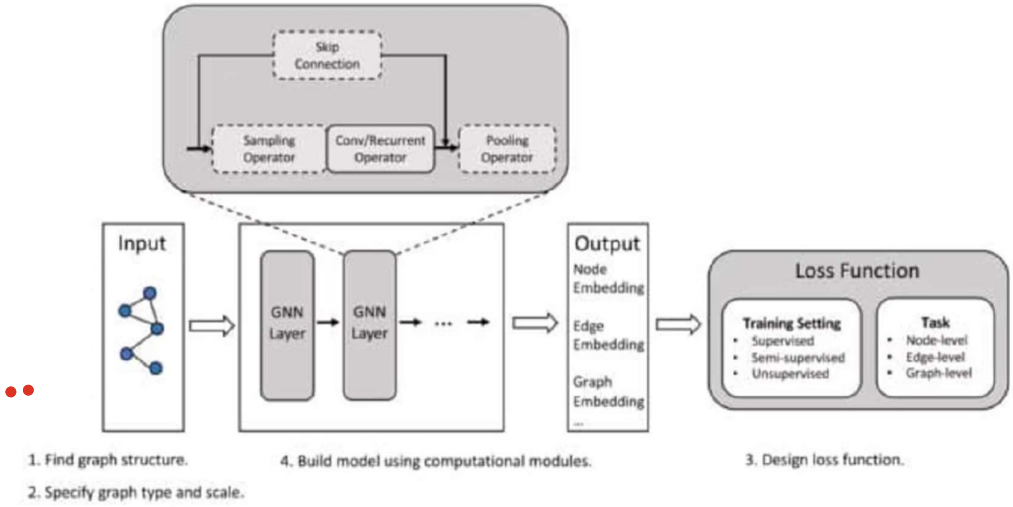


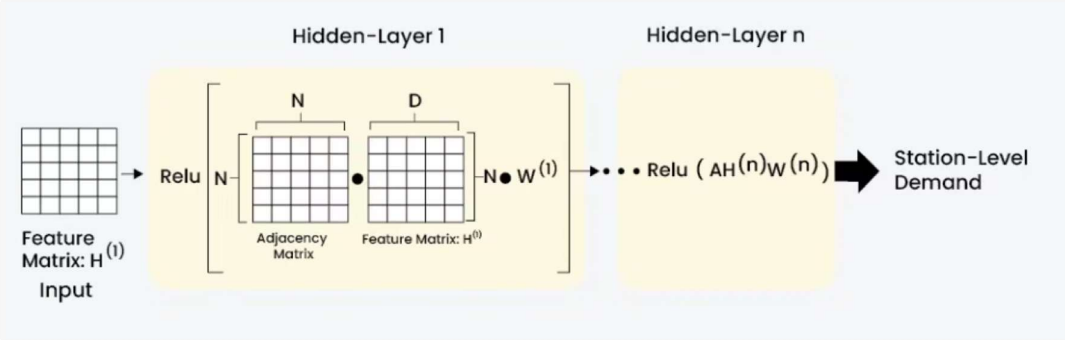
Fig : Pipeline of GNN

What Are Graph Convolutional Networks?

Graph Convolutional Networks (GCNs) are a type of **neural network** designed to work directly with graphs. A graph consists of nodes (vertices and edges (connections between nodes). In a GCN, each node represents an entity, and the edges represent the relationships between these entities. The primary goal of GCNs is to learn node embeddings, which are vector representations of nodes that capture the graph's structural and feature information.

Graph Convolutional Networks (GCNs) are **powerful deep learning models designed to perform convolutions on graph-structured data**, enabling machine learning on non-Euclidean data like social networks, molecules, and citation networks. They update node representations by aggregating features from a node's immediate neighbors, effectively smoothing features across the graph.

Architecture of GCNs



GCNs typically consist of multiple layers, each responsible for refining node embeddings by aggregating information from neighbors at increasing distances. The layers are:

Input Layer: The input layer initializes the node features, usually from raw data or pre-1. trained embeddings.

Hidden Layers: Hidden layers perform the graph convolution operations, progressively aggregating and transforming node features.

2. **A. Graph Convolutional Layers**: These layers perform the convolution operation on the graph. Each layer updates the feature representation of a node by aggregating the features of its neighbors.

B. Activation Functions: Non-linear functions such as ReLU are applied to the output of each convolutional layer to introduce non-linearity into the model.

C. Pooling Layers: These layers reduce the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures.

3. Output Layer: The output layer produces the final node embeddings or predictions, depending on the task (e.g., node classification, link prediction).

4. **Fully Connected Layers**: These layers are used at the end of the network to perform tasks such as classification or regression.

Graph SAGE

GraphSAGE is a framework for inductive representation learning on large graphs. GraphSAGE is used to generate low-dimensional vector representations for nodes, and is especially useful for graphs that have rich node attribute information.

Instead of training individual embeddings for each node, the algorithm learns a function that generates embeddings by sampling and aggregating features from a node's local neighborhood.

GraphSAGE (Graph SAmple and Aggregate) is **an inductive framework for efficient node representation learning on large graphs, developed to generate embeddings for unseen nodes**. Unlike transductive methods that learn fixed embeddings, GraphSAGE learns aggregation functions to generate embeddings by sampling and combining feature information from a node's local neighborhood, allowing for scaling to massive graphs.

Key Features of GraphSAGE

Inductive Capability: GraphSAGE can generate embeddings for new, unseen nodes or entirely new graphs, making it highly effective for dynamic graphs where new data arrives regularly.

Neighborhood Sampling: To avoid computing full graph information (which is computationally expensive), GraphSAGE samples a fixed-size set of neighbors for each node, which makes it scalable to graphs with millions of nodes.

Aggregation Functions: Instead of using all neighbors, it aggregates features from sampled neighbors. These functions must be order-invariant (e.g., mean, LSTM, or pooling aggregators).

Feature-Based: It leverages node features (attributes) rather than just the graph structure (topology) to generate embeddings.

Core Architecture Components

The GraphSAGE architecture is built on three main pillars: sampling, aggregation, and learning.

Neighbor Sampling: To handle large graphs and avoid the "neighbor explosion" problem, GraphSAGE does not use all neighbors.

Instead, it samples a fixed-size subset of neighbors for each node in each layer.

Aggregation: The model aggregates feature information from the sampled neighbors using differentiable, permutation-invariant aggregation functions.

Update/Concatenation: The aggregated neighbor vector is combined with the node's current representation, followed by a nonlinear transformation to produce the next-layer embedding.

Graph Attention Networks (GAT).

Graph Attention Networks are **a type of neural network architecture that apply attention mechanisms to graph-structured data**.

They allow nodes to assign different levels of importance (attention weights) to their neighbors, enabling more flexible and adaptive aggregation of information compared to fixed convolutional operators. GATs use multi-head attention to improve model stability and capacity.

Key Aspects of Graph Attention Networks:

Attention Mechanism: GATs compute the weight (attention coefficient) for each edge, determining how much a node should attend to its neighbor. These weights are computed using a trainable function (often a small neural network) and normalized across neighborhoods using a softmax function.

Multi-Head Attention: To stabilize the learning process, GATs use multi-head attention, which computes several independent attention mechanisms in parallel, which are then concatenated or averaged.

Node Representation Update: The new representation (embedding) for a node is a weighted sum of the transformed features of its neighbors.

Inductive & Transductive Capabilities: GATs can be applied to both node classification (transductive) and graph classification (inductive) tasks, even allowing for generalization to unseen graphs.

Efficiency: The operation is highly parallelizable across all edges in the graph, making it computationally efficient.

Differences from GCNs:

Unlike Graph Convolutional Networks (GCNs), which use fixed weights (like the graph Laplacian) based on node degrees, GATs compute attention weights that depend on the features of the nodes involved, allowing for more nuanced model.

Graph Isomorphism Network (GIN)

Graph Isomorphism Networks (GIN) are **a powerful class of Graph Neural Networks (GNNs) designed to maximize discriminative power**, making them as effective as the 1-dimensional Weisfeiler-Lehman (1-WL) graph isomorphism test. GIN achieves this by using an injective, sum-based aggregation function and MLPs to update node features, ensuring that structurally different graphs are mapped to unique representations

How GIN Works

The "magic" of GIN lies in its **injective** aggregation and update functions. In simple terms, it ensures that if two nodes have different types or numbers of neighbors, they will always end up with different mathematical representations.

Sum Aggregator: Unlike other models that might average their neighbors' features (which can lose information about how many neighbors there are), GIN uses a **sum**. If one node has two "blue" neighbors and another has three, a sum will distinguish them, whereas a mean might not.

MLP Update: After summing neighbor features, GIN passes the result through a **Multi-Layer Perceptron (MLP)**. This is a universal function approximator that can learn to map different inputs to unique outputs.

$$h_v^{(k)} = MLP^{(k)}\left(\left(1 + \epsilon^{(k)}\right) \cdot h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)} \right)$$

Here, is a parameter (often fixed at 0 or learned) that weights a node's own feature against its neighbors' ϵ features.

Why It Matters

Maximal Power: It is theoretically proven to be the most expressive message-passing GNN possible.

Graph Classification: Because it is so good at distinguishing structures, it is a standard choice for tasks where the "shape" of the graph is critical, such as predicting the properties of **molecules** in drug discovery.

Interpretability: GIN is often used in neuroscience to analyze brain connectivity (fMRI data) because its structure is similar to traditional convolutions, making it easier to visualize which parts of the brain are most important for a prediction.

Spectral Graph Convolution (Chebyshev networks)

Spectral Graph Convolutions using Chebyshev Networks (ChebNet) **represent a significant advancement in graph neural networks (GNNs) by enabling fast, localized spectral filtering on graphs without requiring expensive eigen-decomposition of the graph Laplacian.**

ChebNet approximates spectral filters by a truncated expansion of Chebyshev polynomials up to a specified order , which restricts

the convolution to a K-hop neighborhood.

Key Architecture Components

Localized Spectral Filters: Instead of using the entire graph Fourier basis, ChebNet filters are parameterized as a polynomial of the Laplacian of order K-1, where is the filter support size K (number of hops).

Truncated Chebyshev Polynomials: The convolution is approximated using a Chebyshev polynomial expansion (Ti(L))up to order K, which defines a strict $K - 1$-hop neighborhood filter.

Linear Complexity: The recursive definition of Chebyshev polynomials allows for efficient calculation, making the complexity of the operation linear with respect to the number of edges.

Graph Pooling: ChebNet includes a specific graph pooling method (e.g., Graclus) that groups nodes to build a coarser graph, allowing for multi-resolution feature extraction.

Chebyshev Convolutional Operation

The spectral convolution $g_\theta \star x$ is approximated by:

$$g_\theta \star x = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L}) x$$

Where:

- $\theta \in \mathbb{R}^K$ is the vector of learnable coefficients.
- $T_k(\tilde{L})$ is the Chebyshev polynomial of order k applied to the scaled Laplacian $\tilde{L}$.
- $\tilde{L} = \frac{2}{\lambda_{\max}} L - I_N$ is the rescaled Laplacian, ensuring eigenvalues are in $[-1, 1]$

Temporal & Dynamic Graph Modeling

Temporal Graph Modeling Architectures are **specialized machine learning frameworks designed to process graphs where nodes, edges, and features evolve over time**. Unlike static Graph Neural Networks (GNNs), these models

capture both complex structural relationships and temporal dependencies (e.g., node interactions, edge appearance/disappearance) to enable predictive tasks such as link prediction and node classification

Core Components of TGN Architecture

TGNs, designed by Twitter researchers, consist of four main components:

1. **Memory Module**: Tracks the temporal evolution of each node, acting as a compressed representation of its past interactions.
2. **Message Generation & Aggregation**: When a new event (interaction) occurs, a message is generated. The aggregation function combines multiple messages for a node within a batch.

3. **Memory Updater**: Updates the node's memory based on the aggregated messages (often using recurrent units like RNN).

4. Temporal Embedding: Generates node embeddings that reflect the current graph structure and the node's recent temporal history, avoiding the issue of "stale memory".

Taxonomies of Temporal Graph Architectures

Temporal graph models are classified based on how they handle time and structure:

Continuous-Time Dynamic Graphs (CTDG): Models edge events as a stream of events with fine-grained timestamps (e.g., TGN,TGAT, JODIE).

Discrete-Time Dynamic Graphs (DTDG): Models the graph as a series of static snapshots, using sequences of GNNs combined with recurrent networks (RNNs/LSTMs) or transformers (e.g., ROLAND).

Temporal Graph Transformers: Use attention mechanisms over time and graph structures (e.g., T3former, DyGFormer) to capture long-range dependencies.

State Space Models (SSMs): Emerging approaches like GraphSSM provide efficient, scalable modeling of extremely long sequences in graphs, offering better long-term dependencies than RNNs.

Oversmoothing and Oversquashing Issues

Oversmoothing and oversquashing are **two fundamental, often inversely related, limitations in deep Graph Neural Networks (GNNs)**. Oversmoothing occurs when deep layers cause node representations to become indistinguishable, losing discriminative power. Oversquashing happens when excessive aggregation compresses distant information into fixed-length vectors, creating bottleneck failures, particularly in graphs with narrow structural connections.

Oversmoothing

Definition: As GNNs stack more layers, node embeddings converge to a similar representation, losing structural and feature-based information, making them indistinguishable.

Cause: Repeated, excessive aggregation (convolution/message passing) forces node features to blend, often leading to performance degradation in deep models (optimal depth is often just 2-5 layers).

Impact: Nodes become too similar, resulting in loss of predictive power.

Mitigation: Using skip connections, residual connections, and restricting network depth.

Oversquashing

Definition: The phenomenon where information from distant nodes is lost or distorted when forced through a bottleneck during aggregation, despite needing that information to make predictions.

Cause: The exponentially growing neighborhood in large graphs is compressed into a fixed-size vector for a central node, creating a structural bottleneck (e.g., small spectral gap).

Impact: The model fails to capture long-range dependencies, failing tasks involving distant nodes.

Mitigation: Using attention mechanisms to make aggregation adaptive, modifying message passing to regulate information flow,

or using, topological modification (e.g., adding shortcuts).

Key Differences and Relationship

Trade-off: Research indicates an "inevitable" trade-off; techniques reducing over-smoothing (like adding layers) often exacerbate over-squashing, while reducing bottlenecks (to fix over-squashing) can accelerate over-smoothing.

Nature: Oversmoothing is about **homogeneity** (representations become too similar), whereas oversquashing is about **information loss** (bottlenecks restrict information propagation).

Detection: Oversmoothing often appears as flat performance or poor accuracy on node classification, while oversquashing appears as poor performance on tasks requiring long-range communication, despite sufficient layers.

Large-Graph Sampling Techniques

Large-graph sampling techniques **reduce massive graphs into representative subgraphs for analysis or Machine Learning (GNNs), balancing computational efficiency with structural accuracy**. Key methods include node/edge-based random sampling, exploration-based methods (e.g., Random Walks, Forest Fire), and subgraph-based approaches (e.g., Cluster-GCN).

1. Randomized Selection Methods

These methods directly select a fraction of nodes or edges from the original graph, often using simple random sampling (uniform).

Node Sampling (Induced Subgraph): Uniformly selects nodes and all edges between them from *n* the original graph.

Edge Sampling: Selects edges uniformly at random and includes their associated nodes.

SNAPE (Sample Nodes And Pick Edge): Samples nodes with a probability and selects edges among the remaining sample, effective for finding maximum matching.

2. Exploration-Based Methods (Traversal)

These methods explore the graph's neighborhood, making them better at maintaining the community structure of the original graph.

Random Walk Sampling: Performs a random walk to visit nodes, which can be weighted to avoid getting stuck in small components.

Forest Fire Sampling: Mimics the spread of fire, where each selected node "burns" its neighbors, which in turn burn their neighbors, creating connected clusters.

Random Node Neighbor (RNN): Selects a node uniformly and explores its immediate neighborhood (out-going neighbors).

3.Subgraph Based Methods (Clustering)

These methods involve partitioning the graph into several smaller clusters or subgraphs, often used for training Graph Neural Networks (GNNs).

Cluster-GCN: Partitions the graph into distinct clusters (e.g., using METIS) and uses these clusters as batches, reducing the computational graph size significantly.

GraphSAINT: Samples the nodes/edges to construct full induced subgraphs, offering high scalability for deep learning.

4.Neighbourhood Sampling (For GNN)

Used in architectures like GraphSAGE to manage computational complexity during message passing

Layer-wise Sampling: Samples a fixed number of neighbors at each hop (e.g., neighbor sampling on 1-hop or 2-hop) rather than using the entire neighborhood.

Key Considerations

Bias: Simple node sampling can miss low-degree nodes, while Random Walks can oversample dense communities.

Efficiency: Subgraph sampling methods (like Cluster-GCN) offer better scalability than traversing huge graphs.

Goodness of Sample: Techniques are evaluated based on how well they retain properties like degree distribution, path length, and clustering coefficient.

Edge Sampling: Selects edges uniformly at random and includes their associated nodes.

SNAPE (Sample Nodes And Pick Edge): Samples nodes with a probability and selects edges among the remaining sample,effective for finding maximum matching.

Introduction to Distributed Graph Learning Deep Graph Learning

Distributed graph learning is a technique used to train machine learning models—typically **Graph Neural Networks (GNNs)**—on datasets so large that they cannot fit into the memory or processing capacity of a single computer.

Unlike traditional distributed machine learning, which often assumes data points are independent (like separate images), graph learning must account for the **connections (edges)** between data points.

Why It Is Necessary

Modern graphs, such as social networks, financial transaction webs, or recommendation systems, often contain **billions of nodes and trillions of edges**. Processing these requires a cluster of machines to work together to overcome hardware limitations.

Deep Graph Library (DGL)

Deep Graph Library (DGL) is a framework that allows one to experiment with graph machine learning techniques. It has a very clean and concise API and is an amazing thing to use.

In DGL, distributed graph learning is handled by a specialized stack called DistDGL. It is designed to make a cluster of machines act like one giant GPU/CPU by managing how graph data is stored and accessed across the network. Here is the breakdown of how DGL specifically handles this:

The "Shared-Nothing" Storage (Partitioning)

DGL doesn't store the whole graph on every machine. Instead, it uses Graph Partitioning (usually via the METIS algorithm) to break the graph into pieces.

Each machine (Node) in your cluster stores one Partition.

Each partition includes the local nodes, their edges, and their features.

HALO Nodes: DGL also stores "ghost" copies of nodes that are one hop away in another partition. This allows for local computation without constant networking.

The Three-Process Architecture

When you run a distributed DGL job, three distinct types of processes start on each machine:

1. Servers: These are the "data keepers." they hold the graph structure and node/edge features. They wait for requests to send data to trainers.

2. Samplers: These act as "gatherers." They reach out to servers (local or remote) to sample neighbors and create 2. mini-batches.

3. Trainers: These are the "thinkers." They take the mini-batches from the samplers, run the forward/backward pass, and update the

model weights.

Key DGL Distributed Components

DGL provides specific "Dist" versions of standard objects to hide the complexity of the network:

1. DistGraph: You treat this like a normal DGL graph object, but behind the scenes, it knows which nodes are local and which ones it needs to fetch from another machine.

2. DistTensor: This allows you to store massive feature matrices (like 100GB of node embeddings) across the RAM of multiple machines.

3. DistDataLoader: This replaces the standard DataLoader; it ensures each trainer only processes the nodes assigned to its specific partition to avoid redundant work.

The Workflow

1. Preprocessing: You run a script to partition the graph and save it to a shared filesystem (like NFS or S3).

2. Launching: You use the dgl.distributed.launch utility. It uses SSH into your machines and starts the Servers and Trainers.

3. Training: Trainers request mini-batches. If a trainer needs a neighbor that is on a different machine, the DGL KVStore (Key-Value Store) automatically fetches it over the network.

Why use DGL for this?

The main advantage is transparency. Your training loop code looks almost identical to a local script; DGL handles the socket communication, data serialization, and load balancing in the background.